ITCS496 PHYSICAL IMPLEMENTATION OF DATABASES
Semester I, 2025-2026

TERM PROJECT – Handout#1

Throughout the remainder of the semester, you are required to develop an Oracle database application as a term project. The aim of the project is to practice actual database system development.

This and other project specification documents will serve as an essential guideline while you are developing your term project. So, please read it carefully and keep it throughout the semester. You can also use these documents as a *checklist* before submitting a report, i.e., at every major stage of the project, to see if you have satisfied the basic requirements of that particular stage. You should also check announcements on the Blackboard course page regularly to be informed on the project status.
The general issues about this term project are listed below:

- First, form project groups of **4-6 students**.
- Note that we are assuming that the design of the database system is over, and we are in the stage of the physical implementation of the system.
- Do a project demonstration During **Dec 7th, 2025 – Dec 14th, 2025**
- Submit a complete project (both a report and development code) by **Sunday** **Dec 14th, 2025**, **at 23:59**.

Employee Management System

The **Employee Management System** (EMS) is a centralized platform designed to manage employee data, monitor attendance, handle payroll processes, track performance, and streamline administrative operations within an organization. The system provides role-based access and ensures data security, accuracy, and automation of HR workflows.

Project Objectives

2. Objectives of the System

- Improve data accuracy and organization
- Automate employee and HR processes
- Provide a self-service portal for employees
- Allow administrators to manage system-level configurations
- Facilitate attendance monitoring and payroll calculation
- Improve communication between employees and HR
- Generate reliable reports for decision-making

3. User Types and Their Functionalities

A. Administrator : The Administrator has the highest level of control over the system.

Main Functionalities:

User & Role Management

- Create, update, deactivate user accounts
- Assign roles: Admin, HR, Employee

Data Management

- Manage departments, job titles, and employment levels
- Manage organization hierarchy

Reports

- Employee statistics
- HR productivity reports
- Full system audit log

B. HR Manager

The HR Manager manages operations related to employees, attendance, and payroll.

Employee Management

- Register new employees
- Update personal and employment information
- Terminate or deactivate employee profile

Attendance & Leave Management

- View attendance logs (daily, weekly, monthly)
- Approve or reject employee leave requests
- Mark manual attendance adjustments
- Monitor late/early clock-ins

Payroll Management

- Enter monthly salaries and allowances
- Generate payroll based on attendance & deductions
- Produce payslips for employees

Reports

- Employee list
- Attendance summary
- Payroll summary
- Leave usage report
- Department-wise performance

C. Employee: Employees have access to their self-service portal.

Personal Profile

- View personal information
- Update contact details

Attendance

- View monthly attendance
- View late/early hours
- View overtime details

Leave Management

- Apply for leave
- View leave balance
- Track request status

Payroll

- View payslips
- View salary breakdown
- View deductions and bonuses

High-Level System Workflow

Employee Life Cycle

1. HR registers employee
2. Admin assigns system account
3. Employee starts attendance & workflows
4. Employee applies for leave
5. HR approves leave
6. System generates monthly payroll
7. Admin reviews reports

Reports (Examples)

For Admin

- System usage report
- Department workforce comparison
- HR performance report

For HR

- Monthly attendance
- Leave balance summary
- Salary breakdown
- Department performance graph

For Employee

- Personal attendance summary
- Leave balance
- Payslip history

Sequences:

Most of the tables will require their own sequence to automatically generate unique primary keys. Make sure that you create such sequences and later use them in your code in INSERT statements. Add triggers if needed to enforce the generation.

Tasks to be done

Each project team is required to develop EMS with varying levels of advanced features with varying levels of advanced features using Oracle APEX.

Part 1: Pending

Part 2: APEX application development:

Full Implementation of the database application for the three different users using Oracle APEX exploiting advanced features. Details of the minimum needed functionalities will be posted soon.

Note also that these are just examples of operations that they can do. You can add more operations if they make sense in such an environment.

Work to be submitted

1. All code written during development (SQL statements, Form triggers, Functions, Procedures, PL/SQL scripts, … etc.).
2. APEX Application.
3. A typed report that contains the following:
   a) Title page that shows your name, ID and section number on the first page of the report.
   b) Assumptions made by you while building the database application.
   c) An ER diagram of the system.
   d) ERD to relation schema mapping
   e) Some screen captures of query/procedure results.
   f) Screen captures of different Forms and Reports with explanation.

Grading Criteria

Your grade will be based upon the following components:

1. The completeness and quality of the database system.
2. Thorough analysis.
3. Clarity and organization of the final report.
4. Project demo/presentation (all students must be available during demo time).
5. Participation (i.e. each team member must know all of the components in the designed database system and can answer the questions)

Important Notes

1. You must submit a softcopy of your report on the due date and time on BB. You will be asked also to present a demo of your database system.
2. If you do not submit your report on the due date, there will be a penalty of four marks for submitting the work on the next day. NO report will be accepted after that and you will be given Zero in this assessment.
3. Work copied from anybody else will be treated very seriously and F/0 will be given in the project.

Tables and Relationships:

1. User & Security Schema

1.1 EMS_ROLES

Column	Type	Description
ROLE_ID (PK)	NUMBER	Unique role id
ROLE_NAME	VARCHAR2(50)	Administrator, HR, Employee
DESCRIPTION	VARCHAR2(255)	Role purpose

1.2 EMS_USERS

Column	Type	Description
USER_ID (PK)	NUMBER	Unique system user
USERNAME	VARCHAR2(50)	Login username
PASSWORD	VARCHAR2(25)	password
EMAIL	VARCHAR2(255)	User email
IS_ACTIVE	CHAR(1)	Y/N

1.3 EMS_USER_ROLES

Column	Type	Description
USER_ROLE_ID (PK)	NUMBER	Mapping ID
USER_ID (FK → EMS_USERS.USER_ID)	NUMBER	A user
ROLE_ID (FK → EMS_ROLES.ROLE_ID)	NUMBER	Role for user
ASSIGNED_AT	TIMESTAMP	timestamp

Relationship:

- A user can have **multiple roles**
- A role can be assigned to **many users**

2. Organizational Structure Schema

2.1 EMS_DEPARTMENTS

Column	Type
DEPARTMENT_ID (PK)	NUMBER
DEPT_CODE	VARCHAR2(20)
DEPT_NAME	VARCHAR2(100)
MANAGER_ID (FK → EMS_EMPLOYEES.EMPLOYEE_ID)	NUMBER
CREATED_AT	TIMESTAMP

2.2 EMS_POSITIONS

Column	Type
POSITION_ID (PK)	NUMBER
TITLE	VARCHAR2(100)
LEVEL	VARCHAR2(50)
JOB_DESCRIPTION	CLOB

3. Employee Schema

3.1 EMS_EMPLOYEES

Column	Type
EMPLOYEE_ID (PK)	NUMBER
EMPLOYEE_CODE	VARCHAR2(30)
FIRST_NAME	VARCHAR2(100)
LAST_NAME	VARCHAR2(100)
GENDER	VARCHAR2(10)
DATE_OF_BIRTH	DATE

Column	Type
HIRE_DATE	DATE
EMAIL	VARCHAR2(255)
PHONE_NUMBER	VARCHAR2(20)
ADDRESS	VARCHAR2(400)
DEPARTMENT_ID (FK)	NUMBER
POSITION_ID (FK)	NUMBER
EMPLOYMENT_STATUS	VARCHAR2(20)
SALARY_BASE	NUMBER(15,2)
CREATED_AT	TIMESTAMP

Relationships:

- Employee belongs to *1 department*
- Employee holds *1 position*

3.2 EMS_EMPLOYMENT_HISTORY

Column	Type
HISTORY_ID (PK)	NUMBER
EMPLOYEE_ID (FK)	NUMBER
START_DATE	DATE
END_DATE	DATE
POSITION_ID (FK)	NUMBER
DEPARTMENT_ID (FK)	NUMBER
SALARY_AT_TIME	NUMBER

Purpose:
Tracks promotions, transfers, etc.

4. Attendance Schema

4.1 EMS_WORK_SHIFTS

Column	Type
SHIFT_ID (PK)	NUMBER
SHIFT_NAME	VARCHAR2(100)
START_TIME	VARCHAR2(10)
END_TIME	VARCHAR2(10)
GRACE_MINUTES	NUMBER
NOTES	VARCHAR2(400)

4.2 EMS_ATTENDANCE_LOGS

Column	Type
ATTENDANCE_ID (PK)	NUMBER
EMPLOYEE_ID (FK)	NUMBER
SHIFT_ID (FK)	NUMBER
CHECK_IN	TIMESTAMP
CHECK_OUT	TIMESTAMP
DURATION_MINUTES	GENERATED
SOURCE	VARCHAR2(50)
NOTE	VARCHAR2(400)

5. Leave Management Schema

5.1 EMS_LEAVE_TYPES

Column	Type
LEAVE_TYPE_ID (PK)	NUMBER
CODE	VARCHAR2(20)
NAME	VARCHAR2(100)
DESCRIPTION	VARCHAR2(400)
DEFAULT_DAYS	NUMBER
PAID_LEAVE	CHAR(1)

5.2 EMS_LEAVE_REQUESTS

Column	Type
LEAVE_ID (PK)	NUMBER
EMPLOYEE_ID (FK)	NUMBER
LEAVE_TYPE_ID (FK)	NUMBER
START_DATE	DATE
END_DATE	DATE
DAYS_REQUESTED	NUMBER
REASON	CLOB
STATUS	VARCHAR2(20)
REQUESTED_AT	TIMESTAMP
DECIDED_BY	NUMBER
DECIDED_AT	TIMESTAMP

6. Payroll Schema

6.1 EMS_PAY_ALLOWANCES

Column	Type
ALLOWANCE_ID (PK)	NUMBER
NAME	VARCHAR2(100)
AMOUNT	NUMBER
TAXABLE	CHAR(1)

6.2 EMS_PAY_DEDUCTIONS

Column	Type
DEDUCTION_ID (PK)	NUMBER
NAME	VARCHAR2(100)
AMOUNT	NUMBER
PERCENTAGE	NUMBER

6.3 EMS_PAY_SALARIES

Column	Type
PAY_ID (PK)	NUMBER
EMPLOYEE_ID (FK)	NUMBER
PAYROLL_MONTH	DATE
BASE_SALARY	NUMBER
GROSS_AMOUNT	NUMBER
TOTAL_DEDUCTIONS	NUMBER
NET_AMOUNT	NUMBER
STATUS	VARCHAR2(20)
PROCESSED_AT	TIMESTAMP

6.4 EMS_PAYSLIPS

Column	Type
PAYSLIP_ID (PK)	NUMBER
PAY_ID (FK)	NUMBER
EMPLOYEE_ID (FK)	NUMBER
ISSUE_DATE	DATE
GROSS_AMOUNT	NUMBER
DEDUCTIONS	NUMBER
NET_AMOUNT	NUMBER
XML_PAYLOAD	CLOB

8. Audit Schema

8.1 EMS_AUDIT_LOG

Column	Type
AUDIT_ID (PK)	NUMBER
USER_ID (FK)	NUMBER
ACTION	VARCHAR2(100)
OBJECT_TYPE	VARCHAR2(100)
OBJECT_ID	VARCHAR2(100)
OLD_VALUE	VARCHAR2(1000)
NEW_VALUE	VARCHAR2(1000)
ACTION_TIME	TIMESTAMP

Summary of Relationships

Table	Related To	Relationship
USERS ↔ ROLES	USER_ROLES	Many-to-many
EMPLOYEES ↔ DEPARTMENTS	FK	Many employees per department
EMPLOYEES ↔ POSITIONS	FK	Many employees per position
EMPLOYEES ↔ ATTENDANCE_LOGS	FK	One-to-many
EMPLOYEES ↔ LEAVE_REQUESTS	FK	One-to-many
EMPLOYEES ↔ PAY_SALARIES	FK	One-to-many
PAY_SALARIES ↔ PAYSLIPS	FK	One-to-one
EMPLOYEES ↔ EMPLOYMENT_HISTORY	FK	One-to-many
LEAVE_TYPES ↔ LEAVE_REQUESTS	FK	One-to-many